

Exercise 2 — Fixing Production Issues

You're the engineer after the incident

Goal: Apply fixes to the vulnerable API — each maps to a vulnerability you broke in Exercise 1. Open the API source in your editor. After each fix, run `./gradlew test` to verify it instantly.

Fixes 1–3: Core — everyone should reach Fix 3. **Fixes 4–8:** ⌚ if time — fast groups can work through all of them.

Setup

- ☐ API is **stopped**: `docker compose down`
- ☐ Open the `src/` folder in your IDE or editor
- ☐ After each fix, run `./gradlew test` to verify, then `docker compose up --build` to test with Postman

Fix 1 — Add Ownership Checks (IDOR) ⌚ 25 min

1a — AccountService + AccountController

File: `src/main/java/com/example/wallet/service/AccountService.java`

Before:

```
public Account getAccountById(Long id) {  
    return accountRepository.findById(id)    // ← no ownership check  
        .orElseThrow(() -> new RuntimeException("Account not found"));  
}
```

After:

```
public Account getAccountById(Long id, String callerUsername) {  
    AppUser caller = userRepository.findByUsername(callerUsername)  
        .orElseThrow(() -> new RuntimeException("User not found"));  
    Account account = accountRepository.findById(id)  
        .orElseThrow(() -> new RuntimeException("Account not found"));  
    if (!account.getOwner().getId().equals(caller.getId())) {  
        throw new org.springframework.security.access.AccessDeniedException(  
            "You don't own this account");  
    }  
    return account;  
}
```

File: `src/main/java/com/example/wallet/controller/AccountController.java`

After — add `Authentication auth` and pass it through:

```
@GetMapping("/{id}")
public ResponseEntity<Account> getAccountById(
    @PathVariable Long id, Authentication auth) {
    return ResponseEntity.ok(accountService.getAccountById(id, auth.getName()));
}
```

1b — UserService + UserController

File: src/main/java/com/example/wallet/service/UserService.java

Before:

```
public UserResponse getUserById(Long id) {
    AppUser user = userRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("User not found"));
    return new UserResponse(user.getId(), user.getUsername(),
        user.getEmail(), user.getPassword(), user.getRole());
}
```

After:

```
public UserResponse getUserById(Long id, String callerUsername) {
    AppUser caller = userRepository.findByUsername(callerUsername)
        .orElseThrow(() -> new RuntimeException("User not found"));
    AppUser user = userRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("User not found"));
    if (!user.getId().equals(caller.getId())) {
        throw new org.springframework.security.access.AccessDeniedException(
            "You don't own this profile");
    }
    return new UserResponse(user.getId(), user.getUsername(), user.getEmail(),
        user.getRole());
}
```

File: src/main/java/com/example/wallet/controller/UserController.java

After — add Authentication auth and pass it through:

```
@GetMapping("/{id}")
public ResponseEntity<UserResponse> getUserById(
    @PathVariable Long id, Authentication auth) {
    return ResponseEntity.ok(userService.getUserById(id, auth.getName()));
}
```

1c — TransactionService + TransactionController

File: src/main/java/com/example/wallet/service/TransactionService.java

Before:

```
public List<Transaction> getTransactionsForAccount(Long accountId) {
    return transactionRepository.findById(accountId);
}
```

After:

```
public List<Transaction> getTransactionsForAccount(Long accountId, String
callerUsername) {
    AppUser caller = userRepository.findByUsername(callerUsername)
        .orElseThrow(() -> new RuntimeException("User not found"));
    Account account = accountRepository.findById(accountId)
        .orElseThrow(() -> new RuntimeException("Account not found"));
    if (!account.getOwner().getId().equals(caller.getId())) {
        throw new org.springframework.security.access.AccessDeniedException(
            "You don't own this account");
    }
    return transactionRepository.findById(accountId);
}
```

File: src/main/java/com/example/wallet/controller/TransactionController.java

After — add `Authentication auth` and pass it through:

```
@GetMapping("/account/{accountId}")
public ResponseEntity<List<Transaction>> getTransactions(
    @PathVariable Long accountId, Authentication auth) {
    return ResponseEntity.ok(
        transactionService.getTransactionsForAccount(accountId,
            auth.getName()));
}
```

 **Tip:** The search endpoint `GET /api/transactions/account/{accountId}/search` has the exact same gap. Apply the same ownership check to `searchTransactions()` — the pattern is identical.

Verify with tests:

```
./gradlew test --tests "/*.fix1_*
```

All 5 `fix1_` tests should go green 

Verify with Postman: Re-run "🔥 IDOR – Alice reads Bob's account (id=2)" → 403 Forbidden .

Fix 2 — Remove Sensitive Fields from Responses

File: src/main/java/com/example/wallet/dto/UserResponse.java

The bug:

```
public class UserResponse {
    private Long id;
    private String username;
    private String email;
    private String password; // ← this should never be here
    private String role;
}
```

The fix — remove the `password` field, keep everything else:

```
public class UserResponse {
    private Long id;
    private String username;
    private String email;
    private String role; // ← keep this
                        // password field removed entirely
}
```

Verify with tests:

```
./gradlew test --tests "/*.fix2_*
```

Verify with Postman: Run "My Profile (Alice – OK)" → `password` field is gone from the response ✓

Fix 3 — Add Security Headers 🕒 10 min

File: `src/main/java/com/example/wallet/config/SecurityConfig.java`

The bug:

```
.headers(headers -> headers.disable()) // ← all headers stripped
```

Step 1 — Add these two imports at the top of the file:

```
import org.springframework.security.config.Customizer;
import org.springframework.security.web.header.writers.ReferrerPolicyHeaderWriter;
```

Step 2 — Replace the `.headers(...)` block inside `filterChain()` :

```
.headers(headers -> headers
    .httpStrictTransportSecurity(hsts -> hsts
        .includeSubDomains(true)
        .maxAgeInSeconds(31536000))
    .frameOptions(frame -> frame.deny())
    .contentTypeOptions(Customizer.withDefaults())
    .referrerPolicy(referrer -> referrer
```

```
        .policy(ReferrerPolicyHeaderWriter.ReferrerPolicy
            .STRICT_ORIGIN_WHEN_CROSS_ORIGIN))
    )
```

Verify with tests:

```
./gradlew test --tests "/*.fix3_*
```

Verify with Postman: Run "Check Security Headers (after fix)" → all 3 tests pass ✓

Fix 4 — Fix CORS to Allowlist Only ⌚ if time — 10 min

File: `src/main/java/com/example/wallet/config/SecurityConfig.java`

The bug:

```
config.setAllowedOriginPatterns(List.of("*")); // ← any origin
config.setAllowCredentials(true);             // ← with credentials = broken
```

The fix:

```
config.setAllowedOrigins(List.of(
    "http://localhost:3000", // local dev frontend
    "https://yourapp.com"   // production frontend
));
config.setAllowCredentials(true);
```

Verify with tests:

```
./gradlew test --tests "/*.fix4_*
```

Verify with Postman: Run "CORS Preflight from evil-site.com" → no Access-Control-Allow-Origin header in the response ✓

Fix 5 — Webhook HMAC Verification ⌚ if time — 15 min

File: `src/main/java/com/example/wallet/controller/WebhookController.java`

The bug — signature header is ignored:

```
@PostMapping("/payment")
public ResponseEntity<?> receivePaymentWebhook(
    @RequestHeader(value = "X-Signature-256", required = false) String
    signature,
    @RequestBody WebhookPayload payload) {
```

```
// signature is never checked
}
```

The fix — add a verification method and call it:

```
@PostMapping("/payment")
public ResponseEntity<?> receivePaymentWebhook(
    @RequestHeader(value = "X-Signature-256") String signature,
    @RequestBody String rawBody) {

    verifySignature(signature, rawBody);

    WebhookPayload payload = parsePayload(rawBody);
    // process payload...
    return ResponseEntity.ok(Map.of("status", "accepted"));
}

private void verifySignature(String received, String body) {
    try {
        Mac mac = Mac.getInstance("HmacSHA256");
        mac.init(new SecretKeySpec(webhookSecret.getBytes(), "HmacSHA256"));
        String expected = "sha256=" + HexFormat.of()
            .formatHex(mac.doFinal(body.getBytes()));

        // ⚠ Critical: use constant-time comparison to prevent timing attacks
        if (!MessageDigest.isEqual(expected.getBytes(), received.getBytes())) {
            throw new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Invalid
signature");
        }
    } catch (NoSuchAlgorithmException | InvalidKeyException e) {
        throw new RuntimeException("HMAC verification failed", e);
    }
}
```

Verify with tests:

```
./gradlew test --tests "/*.fix5_*
```

Verify with Postman: Run "🔥 Fake Webhook (no signature)" → 401 Unauthorized ✅

Fix 6 — Hash Passwords with BCrypt 🕒 if time — 5 min

File 1: src/main/java/com/example/wallet/config/SecurityConfig.java

The bug:

```
// ⚠ NoOpPasswordEncoder stores and compares passwords in plain text
public PasswordEncoder passwordEncoder() {
```

```
        return NoOpPasswordEncoder.getInstance();
    }
```

The fix:

```
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(12);
}
```

File 2: `src/main/java/com/example/wallet/service/AuthService.java`

Add `PasswordEncoder passwordEncoder` as a constructor-injected field, then replace the comparison:

```
// Before – plain-text comparison:
if (!user.getPassword().equals(request.getPassword())) { ... }

// After – constant-time hash comparison:
if (!passwordEncoder.matches(request.getPassword(), user.getPassword())) { ... }
```

Also change both `RuntimeException` throws to

`ResponseStatusException(HttpStatus.UNAUTHORIZED, ...)` so wrong credentials return `401` instead of `500`.

File 3: `src/main/resources/data.sql`

Replace plain-text passwords with `BCrypt(12)` hashes:

```
INSERT INTO app_user (id, username, email, password, role) VALUES
(1, 'alice', 'alice@example.com',
'$2a$12$WBL.sxiZJtgueFyJjFg0y0x3M/oEjrzUbUxfIULpN2L7AXQkHo45.', 'USER'),
(2, 'bob', 'bob@example.com',
'$2a$12$.3CVy1cVLU/Ejowpa6l.R0Bd3n/HPcZIuxrMvXm4sHPz/jvQzFFmC', 'USER'),
(3, 'charlie', 'charlie@example.com',
'$2a$12$PDK4YPgleNlMnIxH9TNJI.0z/epJq1Dmsv7vl8jVA5Sjzw2zcSAIu', 'USER'),
(4, 'admin', 'admin@example.com',
'$2a$12$0a4w70HkyT3P2U0nqwJ8I.RyUCCbZSrVsCfY6.UdbJFe9HFnxpbWu', 'ADMIN');
```

Verify with tests:

```
./gradlew test --tests "/*.fix6_*
```

Fix 7 — Lock Down the Actuator 🕒 if time — 5 min

File: `src/main/java/com/example/wallet/config/SecurityConfig.java`

The bug:

```
.requestMatchers("/actuator/**").permitAll() // ← metrics, env, heap dumps – all public
```

The fix — keep `/health` public (load balancers need it), require auth for everything else:

```
.requestMatchers("/actuator/health").permitAll()  
.requestMatchers("/actuator/**").authenticated()
```

Verify with tests:

```
./gradlew test --tests "/*.fix7_*
```

Fix 8 — Rate Limit the Login Endpoint ⌚ if time — 10 min

File 1: `build.gradle` — add Bucket4j:

```
implementation 'com.bucket4j:bucket4j-core:8.10.1'
```

File 2: Create `src/main/java/com/example/wallet/filter/RateLimitFilter.java` :

```
@Component  
public class RateLimitFilter extends OncePerRequestFilter {  
  
    private final ConcurrentHashMap<String, Bucket> buckets = new  
    ConcurrentHashMap<>();  
  
    @Override  
    protected boolean shouldNotFilter(HttpServletRequest request) {  
        return !(request.getMethod().equals("POST")  
            && request.getRequestURI().equals("/api/auth/login"));  
    }  
  
    @Override  
    protected void doFilterInternal(HttpServletRequest request,  
        HttpServletResponse response,  
        FilterChain filterChain) throws  
        ServletException, IOException {  
        String ip = Optional.ofNullable(request.getHeader("X-Forwarded-For"))  
            .map(xff -> xff.split(",")[0].trim())  
            .orElse(request.getRemoteAddr());  
  
        Bucket bucket = buckets.computeIfAbsent(ip,  
            k -> Bucket.builder()  
                .addLimit(Bandwidth.classic(5, Refill.intervally(5,  
                    Duration.ofMinutes(1))))  
                .build());  
    }  
}
```

```

        if (bucket.tryConsume(1)) {
            filterChain.doFilter(request, response);
        } else {
            response.setStatus(429);
            response.setContentType("application/json");
            response.addHeader("Retry-After", "60");
            response.getWriter().write("{\"error\":\"Too many login attempts. Try again later.\"}");
        }
    }
}

```

Verify with tests:

```
./gradlew test --tests "/*.fix8_*
```

🎯 Debrief — What we fixed

Fix	Technique	Impact
1 — Ownership checks	Authorization in service layer	Stops IDOR attacks
2 — Remove password from DTO	Sensitive data control	Stops credential leakage
3 — Security headers	HTTP response hardening	Stops clickjacking, MIME sniffing
4 — CORS allowlist	Explicit origin control	Stops cross-origin data theft
5 — Webhook HMAC	Constant-time verification	Stops forged payment events
6 — BCrypt hashing	Slow, salted password hash	Stops offline password cracking
7 — Actuator lockdown	Require auth for sensitive endpoints	Stops runtime fingerprinting
8 — Rate limiting	Token bucket per IP (Bucket4j)	Stops brute-force login attacks

Every fix here is a production-ready pattern used in real systems. Not theoretical. Not "best practices." Things that prevent real incidents.